Formalising the Radon-Nikodym Theorem

Yuhan Wang

1 Introduction

For this project, we formalise the Radon-Nikodym theorem, one of the most fundamental results in measure theory. The theorem characterises when a measure ν can be expressed as an integral with respect to another measure μ.

Theorem 1 (Radon-Nikodym)**.** *Let μ and ν be σ-finite measures on a measurable space (X, Σ). Then*

$$\nu \ll \mu \quad \Longleftrightarrow \quad \exists f : X \to [0, \infty], \text{ measurable, such that } \nu = f \cdot \mu.$$

Here $\nu \ll \mu$ denotes *absolute continuity*: $\mu(A) = 0$ implies $\nu(A) = 0$ for all measurable A. The function f is called the *Radon-Nikodym derivative* of ν with respect to μ, written $\frac{d\nu}{d\mu}$.

The Radon-Nikodym theorem is foundational: conditional expectation in probability, likelihood ratios in statistics, and the Riesz representation theorem in functional analysis all rely on it.

The proof requires a chain of intermediate results:

Hahn Decomposition $\Rightarrow$ Jordan Decomposition $\Rightarrow$ Lebesgue Decomposition $\Rightarrow$ Radon-Nikodym.

Lean is an interactive theorem prover that allows mathematical proofs to be written in a formal language and checked by a computer. In this project, I formalise the Radon-Nikodym theorem in Lean 4 using the Mathlib library, building up through the necessary intermediate results.

2 Mathematical Background

2.1 Signed Measures

A *signed measure* on a measurable space (X, Σ) is a σ-additive function $s : \Sigma \to \mathbb{R}$ with $s(\emptyset) = 0$. Unlike positive measures, signed measures can take negative values.

Definition 1 (Positive and Negative Sets)**.** *Let s be a signed measure on (X, Σ).*

- *A measurable set P is* positive *for s if $s(B) \geq 0$ for all measurable $B \subseteq P$.*
- *A measurable set N is* negative *for s if $s(B) \leq 0$ for all measurable $B \subseteq N$.*

2.2 Hahn Decomposition

Theorem 2 (Hahn Decomposition (Theorem 4.13))**.** *For any signed measure α on $(X, \mathcal{A})$, there exist a positive set A and a negative set B such that $X = A \cup B$ and $A \cap B = \emptyset$.*

Proof sketch. Define $b = \inf\{\alpha(B) : B \text{ is a negative set}\}$. Take negative sets B_n with $\alpha(B_n) \to b$; their union $B = \bigcup B_n$ is negative with $\alpha(B) = b > -\infty$. Setting $A = B^c$, one shows A is positive by contradiction: any subset of A with negative measure would contain a negative subset (Lemma 4.16) that could be adjoined to B, contradicting the minimality of b. $\square$

2.3 Jordan Decomposition

Theorem 3 (Jordan Decomposition (Theorem 4.19))**.** *Let α be a signed measure on $(X, \mathcal{A})$. There exists a unique pair (α_+, α_-) of mutually singular (positive) measures on $(X, \mathcal{A})$ such that*

$$\alpha = \alpha_+ - \alpha_-.$$

Proof sketch. Let (A, B) be a Hahn decomposition for α. Define $\alpha_+(E) = \alpha(E \cap A)$ and $\alpha_-(E) = -\alpha(E \cap B)$. Since A is positive and B is negative, both $\alpha_\pm$ are positive measures, and one checks $\alpha_+ - \alpha_- = \alpha$ and $\alpha_+ \perp \alpha_-$ (with witnessing set A). Uniqueness follows from the essential uniqueness of the Hahn decomposition. □

Definition 2 (Total Variation)**.** *The* total variation measure *of a signed measure α is defined as*

$$|\alpha| = \alpha_+ + \alpha_-.$$

2.4 Absolute Continuity and Mutual Singularity

Definition 3 (Absolute Continuity)**.** *A measure ν is* absolutely continuous *with respect to μ, written $\nu \ll \mu$, if*

$$\mu(A) = 0 \implies \nu(A) = 0 \quad \textit{for all measurable } A.$$

Definition 4 (Mutual Singularity)**.** *Two measures μ and ν are* mutually singular*, written $\mu \perp \nu$, if there exists a measurable set A such that $\mu(A) = 0$ and $\nu(A^c) = 0$.*

Lemma 1. *If $\nu \ll \mu$ and $\nu \perp \mu$, then $\nu = 0$.*

Proof. Let A be the set from mutual singularity with $\mu(A) = 0$ and $\nu(A^c) = 0$. For any measurable B:

$$\nu(B) = \nu(B \cap A) + \nu(B \cap A^c) \leq \nu(A) + \nu(A^c) = 0 + 0 = 0,$$

where $\nu(A) = 0$ follows from $\nu \ll \mu$ and $\mu(A) = 0$. □

2.5 Lebesgue Decomposition

Theorem 4 (Lebesgue Decomposition (Theorem 4.20))**.** *Let μ, ν be two (positive) σ-finite measures on $(X, \mathcal{A})$. Then there exists a unique pair of measures (ν_s, ν_{ac}), the* Lebesgue decomposition *of ν with respect to μ, satisfying:*

$$\nu = \nu_s + \nu_{ac}$$

where $\nu_s \perp \mu$ (singular part) and $\nu_{ac}(A) = \int_A f \, d\mu$ for some measurable $f : X \to [0, \infty]$ (absolutely continuous part).

Proof sketch. By σ-finiteness one may reduce to the case where μ and ν are finite. Introduce the set of "dominated densities" $\mathcal{G} = \{g \geq 0 \text{ measurable} : \int_A g \, d\mu \leq \nu(A) \ \forall A\}$, which is closed under pointwise maxima. Taking a maximising sequence and passing to the limit via monotone convergence yields $f \in \mathcal{G}$ with $\int f \, d\mu = \sup_{g \in \mathcal{G}} \int g \, d\mu$. Setting $\nu_s = \nu - f \cdot \mu$, a Hahn-decomposition argument (Lemma 4.22) shows $\nu_s \perp \mu$, and uniqueness follows from the fact that absolute continuity and mutual singularity together force a measure to be zero. □

3 Mathematical Proof of Radon-Nikodym

Direction 1: Density implies Absolute Continuity

Suppose $\nu = f \cdot \mu$ for some measurable $f \geq 0$, meaning $\nu(A) = \int_A f \, d\mu$ for all measurable A. If $\mu(A) = 0$, then

$$\nu(A) = \int_A f \, d\mu = 0,$$

since the integral over a null set is zero. Hence $\nu \ll \mu$.

Direction 2: Absolute Continuity implies Density

Assume $\nu \ll \mu$ with both measures σ-finite. By the Lebesgue decomposition:

$$\nu = \nu_s + \nu_{ac}$$

where $\nu_s \perp \mu$ and $\nu_{ac} = f \cdot \mu$ for some measurable $f \geq 0$.

Since $\nu \ll \mu$, we have $\nu_s \ll \mu$ (as $\nu_s \leq \nu$). But $\nu_s \perp \mu$ by definition of the Lebesgue decomposition. By the key lemma, $\nu_s \ll \mu$ and $\nu_s \perp \mu$ implies $\nu_s = 0$.

Therefore $\nu = \nu_{ac} = f \cdot \mu$, and $f = \frac{d\nu}{d\mu}$ is the Radon-Nikodym derivative.

4 Implementation Discussion

The mathematical development translates into Lean by working within Mathlib's measure theory library. Several design choices in Mathlib differ from standard textbook presentations and affect how the proofs are structured.

Measures take values in $\mathbb{R}^{\infty}_{\geq 0}$. In Mathlib, a measure is a function from measurable sets to $\mathbb{R}\geq 0\infty$ (the type `ENNReal`), which is $[0, \infty]$ equipped with a semiring structure. Addition and multiplication are defined, but subtraction is *truncated*: $a - b = 0$ whenever $a \leq b$. As a consequence, one cannot freely subtract measures or rearrange equalities the way one does with real-valued expressions. To show that a measure value equals zero, the standard idiom is to apply `le_antisymm` with an upper bound and `zero_le _` as the lower bound, establishing equality from both directions. This two-sided pattern appears three times in the key lemma (Part 3) alone.

Mutual singularity is a four-field structure. The type $\nu \perp_m \mu$ (`Measure.MutuallySingular`) carries a separating set A, a proof that A is measurable, a proof that $\nu(A) = 0$, and a proof that $\mu(A^c) = 0$. Destructuring with `rcases` gives direct access to all four components, which is why the key lemma proof can immediately name the set and its properties.

σ-finiteness is a type class. The hypothesis `[SigmaFinite` μ`]` lets Lean's instance resolution automatically synthesise the `HaveLebesgueDecomposition` instance needed by the Lebesgue decomposition. This means that once both measures are declared σ-finite in a `section`, calling `haveLebesgueDecomposition_add` requires no explicit construction—the instance is found automatically.

Absolute continuity is definitional. The notation $\nu \ll \mu$ unfolds to $\forall s,\ \mu(s) = 0 \to \nu(s) = 0$, and the proof term `Iff.rfl` witnesses this (lemma `ac_def`). Applying a hypothesis `hac :` $\nu \ll \mu$ to a proof `hs :` $\mu(s) = 0$ directly returns $\nu(s) = 0$, with no rewriting needed.

Other key notations used throughout:

- `SignedMeasure` α — signed measures on α.
- μ`.withDensity f` — the measure $A \mapsto \int_A f \, d\mu$.

- ν.rnDeriv μ / ν.singularPart μ — the Radon-Nikodym derivative and singular part.
- s.toJordanDecomposition — access to the Jordan decomposition.

The formalisation is split into six parts, mirroring the chain of results in the mathematical development.

What is mine and what comes from Mathlib. Mathlib already contains the full chain from Hahn decomposition through to the Radon-Nikodym derivative, so none of the individual components are new. My contribution is the *organisation*: I state each theorem in the classical form found in the lecture notes (Theorems 4.13, 4.19, 4.20, and Corollary 4.23), assemble them into a single self-contained file that follows the logical chain

$$\text{Hahn} \Rightarrow \text{Jordan} \Rightarrow \text{Lebesgue} \Rightarrow \text{Radon-Nikodym},$$

and fill in the glue proofs that connect library facts. The most substantial original arguments are the key lemma in Part 3 (eq_zero_of_ac_of_mutuallySingular) and the forward direction of radon_nikodym in Part 5: they require splitting sets, estimating measures via the decomposition, and chaining absolute-continuity and mutual-singularity hypotheses in multi-step calc blocks. Parts 1, 2, 4, and 6 are thin wrappers around existing Mathlib lemmas, deliberately kept short to show that the infrastructure is available and to explain the role each result plays.

Part 1 (Hahn Decomposition). The Hahn decomposition theorem asserts that every signed measure s admits a partition of the space into a positive set P and a negative set P^c. The notation 0 ≤[P] s means the restriction of s to P is non-negative on every measurable subset; Lean represents this as a partial order on restricted signed measures. Mathlib already provides the decomposition, so hahn_decomposition_exists is a one-line appeal to exists_compl_positive_negative.

The auxiliary result (Lemma 4.16) is the only Part 1 proof that uses tactics. The Mathlib lemma exists_subset_restrict_nonpos returns its existential fields in the order $(j, meas, subset, neg, val)$, but our statement orders them as $(j, subset, meas, neg, val)$. The obtain tactic destructures the Mathlib result, and exact re-packs the components in the new order.

```
/-- The Hahn Decomposition Theorem (Theorem 4.13). -/
theorem hahn_decomposition_exists :
    ∃ (P : Set α), MeasurableSet P ∧ 0 ≤[P] s ∧ s ≤[Pᶜ] 0 :=
  SignedMeasure.exists_compl_positive_negative s

/-- A set of negative measure contains a negative subset (Lemma 4.16). -/
theorem exists_negative_subset_of_negative_measure {i : Set α} (hi_neg : s i < 0) :
    ∃ j ⊆ i, MeasurableSet j ∧ s ≤[j] 0 ∧ s j < 0 := by
  obtain ⟨j, hj_meas, hji, hj_neg, hj_val⟩ :=
    SignedMeasure.exists_subset_restrict_nonpos hi_neg
  exact ⟨j, hji, hj_meas, hj_neg, hj_val⟩
```

Part 2 (Jordan Decomposition). We define positivePart and negativePart by extracting the components of s.toJordanDecomposition. Both are marked noncomputable because toJordanDecomposition relies on classical choice (the Hahn decomposition is non-constructive). The instance declarations register IsFiniteMeasure with Lean's type class system, so that downstream lemmas requiring a finite measure can use positivePart s without an explicit proof.

Both jordan_decomposition_eq and jordan_parts_mutuallySingular follow the same two-step pattern. First, unfold positivePart negativePart expands our wrapper definitions to

the underlying `toJordanDecomposition.posPart/negPart`, at which point the goal matches the Mathlib lemma exactly. Then `exact` supplies the proof term. In the decomposition identity, `.symm` flips the Mathlib equality $s^+ - s^- = s$ into $s = s^+ - s^-$. The `unfold` step is essential: without it, Lean sees `positivePart s` and `toJordanDecomposition.posPart` as syntactically different, and `exact` fails.

```
noncomputable def positivePart (s : SignedMeasure α) : Measure α :=
  s.toJordanDecomposition.posPart

noncomputable def negativePart (s : SignedMeasure α) : Measure α :=
  s.toJordanDecomposition.negPart

instance (s : SignedMeasure α) : IsFiniteMeasure (positivePart s) :=
  s.toJordanDecomposition.posPart_finite

instance (s : SignedMeasure α) : IsFiniteMeasure (negativePart s) :=
  s.toJordanDecomposition.negPart_finite

/-- s = s⁺ - s⁻ (Theorem 4.19). -/
theorem jordan_decomposition_eq :
    s = (positivePart s).toSignedMeasure - (negativePart s).toSignedMeasure := by
  unfold positivePart negativePart
  exact s.toSignedMeasure_toJordanDecomposition.symm

theorem jordan_parts_mutuallySingular :
    positivePart s ⊥ₘ negativePart s := by
  unfold positivePart negativePart
  exact s.toJordanDecomposition.mutuallySingular

/-- |s| = s⁺ + s⁻. -/
noncomputable def totalVariation (s : SignedMeasure α) : Measure α :=
  positivePart s + negativePart s
```

Part 3 (Absolute Continuity and Mutual Singularity). We first record several basic facts, all proved as proof terms (no `by` blocks):

- `ac_def` uses `Iff.rfl` because $\nu \ll \mu$ is *definitionally* equal to $\forall s,\ \mu(s) = 0 \to \nu(s) = 0$.
- `mutuallySingular_symm` builds the $\leftrightarrow$ with the anonymous constructor $\langle$`.symm, .symm`$\rangle$, supplying one direction in each component.
- `absolutelyContinuous_trans` uses dot notation `hνμ.trans hμρ`, calling the `trans` method on the absolute continuity proof.

We then prove the key lemma: if $\nu \ll \mu$ and $\nu \perp_m \mu$ hold simultaneously, then $\nu = 0$.

```
lemma ac_def : ν ≪ μ ↔ ∀ s, μ s = 0 → ν s = 0 :=
  Iff.rfl

theorem absolutelyContinuous_withDensity (f : α → ℝ≥0∞) :
```

```
    μ.withDensity f ≪ μ :=
  withDensity_absolutelyContinuous μ f

theorem mutuallySingular_symm : μ ⊥ₘ ν ↔ ν ⊥ₘ μ :=
  ⟨Measure.MutuallySingular.symm, Measure.MutuallySingular.symm⟩

theorem absolutelyContinuous_trans {ρ : Measure α}
    (hνμ : ν ≪ μ) (hμρ : μ ≪ ρ) : ν ≪ ρ :=
  hνμ.trans hμρ
```

Key lemma ($\nu \ll \mu$ and $\nu \perp_m \mu$ implies $\nu = 0$). The proof of `eq_zero_of_ac_of_mutuallySingular` is the most substantial manual argument. It proceeds in three stages:

1. `rcases hms with ⟨A, hA_meas, hνA, hμAc⟩` destructures the `MutuallySingular` structure into its four fields. Then `ext s hs` applies measure extensionality, reducing the goal $\nu = 0$ to showing $\nu(s) = 0$ for an arbitrary measurable set s (binding both `s` and its measurability proof `hs`).

2. We split $s = (s \cap A) \cup (s \cap A^c)$. Applying `measure_union` requires a `Disjoint` proof (via `Set.disjoint_iff` and membership destructuring) and a `MeasurableSet` proof for the second set (`hs.inter hA_meas.compl`).

3. The `calc` block chains $\nu(s) = \nu(s \cap A) + \nu(s \cap A^c) = 0 + 0 = 0$. The step to $0 + 0$ uses `congr 1`, which splits the addition into two subgoals—one per summand. The first vanishes by monotonicity ($\nu(s \cap A) \leq \nu(A) = 0$); the second by chaining monotonicity with absolute continuity, using the `ENNReal` pattern described in the preamble.

```
/-- ν ≪ μ and ν ⊥ μ together force ν = 0. From the separating set A (with ν(A) = 0,
μ(Aᶜ) = 0), any measurable s splits as (s ∩ A) ∪ (s ∩ Aᶜ) and both pieces vanish. -/
theorem eq_zero_of_ac_of_mutuallySingular (hac : ν ≪ μ) (hms : ν ⊥ₘ μ) :
    ν = 0 := by
  rcases hms with ⟨A, hA_meas, hνA, hμAc⟩
  ext s hs
  have hsplit : s = (s ∩ A) ∪ (s ∩ Aᶜ) := by rw [inter_union_compl]
  have hdisj : Disjoint (s ∩ A) (s ∩ Aᶜ) := by
    rw [Set.disjoint_iff]
    intro x ⟨⟨_, hxA⟩, ⟨_, hxAc⟩⟩
    exact hxAc hxA
  calc ν s = ν ((s ∩ A) ∪ (s ∩ Aᶜ)) := by rw [← hsplit]
    _ = ν (s ∩ A) + ν (s ∩ Aᶜ) := measure_union hdisj (hs.inter hA_meas.compl)
    _ = 0 + 0 := by
        congr 1
        · exact le_antisymm
            (measure_mono inter_subset_right |>.trans (le_of_eq hνA)) (zero_le _)
        · have hμ : μ (s ∩ Aᶜ) ≤ μ Aᶜ := measure_mono inter_subset_right
          exact hac (hμ.trans (le_of_eq hμAc) |>.antisymm (zero_le _))
    _ = 0 := add_zero 0
```

Part 4 (Lebesgue Decomposition). The forward direction of Radon-Nikodym (Part 5) needs four facts from the Lebesgue decomposition:

(i) the identity $\nu = \nu_s + f \cdot \mu$, to isolate the singular part;

(ii) $\nu_s \perp \mu$, which with the key lemma from Part 3 will kill ν_s;

(iii) measurability of $f = d\nu/d\mu$, required to exhibit a valid density;

(iv) $f \cdot \mu \ll \mu$, used in the reverse direction.

Mathlib constructs the decomposition internally via its `HaveLebesgueDecomposition` type class, which is automatically synthesised for σ-finite measures. We extract these four facts as named lemmas so that Part 5 can cite them directly.

```
theorem lebesgue_decomposition :
    ν = ν.singularPart μ + μ.withDensity (ν.rnDeriv μ) :=
  ν.haveLebesgueDecomposition_add μ

theorem singularPart_mutuallySingular' (μ ν : Measure α) :
    ν.singularPart μ ⊥ₘ μ :=
  ν.mutuallySingular_singularPart μ

theorem rnDeriv_measurable' (μ ν : Measure α) : Measurable (ν.rnDeriv μ) :=
  Measure.measurable_rnDeriv ν μ

theorem withDensity_rnDeriv_ac (μ ν : Measure α) :
    μ.withDensity (ν.rnDeriv μ) ≪ μ :=
  withDensity_absolutelyContinuous μ (ν.rnDeriv μ)
```

Part 5 (Radon-Nikodym Theorem). The goal is a biconditional, so `constructor` splits it into two subgoals (forward and reverse).

Forward direction ($\Rightarrow$). We receive `hac : ν ≪ μ` via `intro`. Then `use ν.rnDeriv μ` provides the existential witness, and `constructor` splits the remaining $\wedge$ into measurability and the identity $\nu = \mu$`.withDensity` f. The Lebesgue decomposition and singular-part lemmas are loaded with named arguments `(μ := μ) (ν := ν)` because both are section variables of the same type, so Lean cannot determine which is which from the goal alone.

To show $\nu_s \ll \mu$, we unfold absolute continuity: `intro s hs` introduces a set s with $\mu(s) = 0$ and we must prove $\nu_s(s) = 0$. The key step is a three-line `calc` showing $\nu_s(s) \leq \nu(s)$:

1. `le_add_of_nonneg_right (zero_le _)` adds the non-negative `withDensity` term;
2. `(Measure.add_apply _ _ _).symm` rewrites pointwise addition as the sum measure applied to s;
3. `rw [← hdecomp]` replaces the sum with ν.

The outer `le_antisymm` closes $\nu_s(s) = 0$ using this bound and $\nu(s) = 0$ from `hac`. With $\nu_s \ll \mu$ and $\nu_s \perp_m \mu$ in hand, the key lemma gives $\nu_s = 0$, and a final `calc` block chains $\nu = \nu_s + f \cdot \mu = 0 + f \cdot \mu = f \cdot \mu$.

Reverse direction ($\Leftarrow$). The tactic `rintro` combines `intro` and `rcases`: it introduces the witness f, discards the measurability proof with _, and names the identity `hf_eq`. After `rw [hf_eq]` rewrites ν to μ`.withDensity` f, the goal is closed by the library lemma `withDensity_absolutelyContinuous`.

```
/-- The Radon-Nikodym Theorem (Corollary 4.23).
(⇐) Immediate from `withDensity_absolutelyContinuous`.
(⇒) Lebesgue gives ν = νₛ + f • μ with νₛ ⊥ μ. Since νₛ ≤ ν ≪ μ we get νₛ ≪ μ,
    and `eq_zero_of_ac_of_mutuallySingular` forces νₛ = 0. -/
theorem radon_nikodym :
    ν ≪ μ ↔ ∃ f : α → ℝ≥0∞, Measurable f ∧ ν = μ.withDensity f := by
  constructor
  · intro hac
    use ν.rnDeriv μ
    constructor
    · exact Measure.measurable_rnDeriv ν μ
    · have hdecomp := lebesgue_decomposition (μ := μ) (ν := ν)
      have hms := singularPart_mutuallySingular' (μ := μ) (ν := ν)
      have hac_singular : ν.singularPart μ ≪ μ := by
        intro s hs
        have h : ν.singularPart μ s ≤ ν s := by
          calc ν.singularPart μ s
              ≤ ν.singularPart μ s + μ.withDensity (ν.rnDeriv μ) s :=
                le_add_of_nonneg_right (zero_le _)
            _ = (ν.singularPart μ + μ.withDensity (ν.rnDeriv μ)) s :=
                (Measure.add_apply _ _ _).symm
            _ = ν s := by rw [← hdecomp]
        exact le_antisymm (h.trans (le_of_eq (hac hs))) (zero_le _)
      have hzero := eq_zero_of_ac_of_mutuallySingular hac_singular hms
      calc ν = ν.singularPart μ + μ.withDensity (ν.rnDeriv μ) := hdecomp
        _ = 0 + μ.withDensity (ν.rnDeriv μ) := by rw [hzero]
        _ = μ.withDensity (ν.rnDeriv μ) := zero_add _
  · rintro ⟨f, _, hf_eq⟩
    rw [hf_eq]
    exact withDensity_absolutelyContinuous μ f
```

Corollaries. If $\nu \ll \mu$ then ν equals the measure built from its own Radon-Nikodym derivative (`withDensity_rnDeriv_eq'`). The derivative is unique μ-a.e.: if two measurable functions define the same measure via `withDensity`, they agree μ-a.e. (`rnDeriv_unique`).

```
theorem withDensity_rnDeriv_eq' (hac : ν ≪ μ) :
    μ.withDensity (ν.rnDeriv μ) = ν :=
  Measure.withDensity_rnDeriv_eq ν μ hac

/-- The Radon-Nikodym derivative is unique μ-a.e. -/
theorem rnDeriv_unique (μ : Measure α) [SigmaFinite μ] {f g : α → ℝ≥0∞}
    (hf : Measurable f) (hg : Measurable g)
    (hf_eq : μ.withDensity f = μ.withDensity g) :
    f =ᵐ[μ] g :=
  (withDensity_eq_iff_of_sigmaFinite hf.aemeasurable hg.aemeasurable).mp hf_eq
```

Part 6 (Applications). We record two standard consequences. Both equalities are stated with `=ᵐ[ρ]`, which is Lean notation for `Filter.EventuallyEq (Measure.ae ρ)`: two functions agree

ρ-almost everywhere, i.e. the set where they differ has ρ-measure zero. The section requires three `[SigmaFinite]` instances (for μ, ν, and ρ) because the chain rule involves three measures; Lean's instance resolution must find all three to apply the Mathlib lemma. Both results are one-line proof terms.

```
/-- Chain rule: dν/dμ · dμ/dρ = dν/dρ  ρ-a.e. -/
theorem rnDeriv_chain (hνμ : ν ≪ μ) :
    ν.rnDeriv μ * μ.rnDeriv ρ =ᵐ[ρ] ν.rnDeriv ρ :=
  Measure.rnDeriv_mul_rnDeriv hνμ

theorem rnDeriv_self : ν.rnDeriv ν =ᵐ[ν] 1 :=
  Measure.rnDeriv_self ν
```

5 Challenges

The most time-consuming step was converting `ENNReal` inequalities into equalities in the key lemma (Part 3). On paper, showing $\nu(s \cap A^c) = 0$ from $\mu(s \cap A^c) \leq \mu(A^c) = 0$ and $\nu \ll \mu$ is one line. In Lean, `measure_mono` returns an inequality $\mu(s \cap A^c) \leq \mu(A^c)$, but the hypothesis `hac : ν ≪ μ` expects an *equality* $\mu(s \cap A^c) = 0$. Bridging this gap required chaining the inequality with `le_of_eq` to reach $\mu(s \cap A^c) \leq 0$, then applying `.antisymm (zero_le _)` to close the two-sided bound. This single conversion—invisible on paper—took more debugging time than the entire paper proof of the lemma, and illustrates how `ENNReal`'s truncated arithmetic (no subtraction, no negation) forces a bounding-then-closing proof style that has no pencil-and-paper analogue.

A second challenge was scoping `[SigmaFinite]` assumptions correctly. I initially wrote the key lemma inside the same `section` as the Lebesgue decomposition (Parts 4–5), which silently added σ-finiteness hypotheses to every result in that section. This was wrong: the key lemma holds for arbitrary measures, and the extra hypotheses would weaken its applicability. Recognising the problem required checking Lean's `#check` output and noticing the unwanted `[SigmaFinite ν]` argument. The fix was to move Part 3 into its own section with only `variable {μ ν : Measure α}`, so that Lean does not insert finiteness assumptions where none are needed.

6 Reflection

Formalising the Radon-Nikodym theorem forced me to confront details that paper proofs leave implicit. The clearest example was the claim "$\nu_s \leq \nu$" in Part 5: on paper this is a throwaway remark, but in Lean it required a three-step `calc` block through the Lebesgue decomposition, and working through it made clear that the inequality rests on the non-negativity of `μ.withDensity`, not on any intrinsic property of the singular part. More broadly, the `ext s hs` tactic made visible how much of measure theory is σ-algebra bookkeeping—every set operation must carry a measurability certificate, a requirement that is entirely invisible in handwritten proofs.

The submitted Lean file compiles without errors on the project toolchain and contains no `sorry` axioms.

Acknowledgements

I used Cursor, an AI-powered code editor backed by large language models from OpenAI, Google, and Anthropic. AI tools assisted with Mathlib library navigation (locating lemma names and un-

derstanding API conventions) and with debugging tactic-level errors. The mathematical choices—which theorems to include, how to structure the proof chain, and the design of each proof—as well as the final writeup are my own work.

References

[1] Walter Rudin. *Real and Complex Analysis*. McGraw-Hill, 3rd edition, 1987.

[2] Paul R. Halmos. *Measure Theory*. Springer, 1974.

[3] P.F. Rodriguez. *MATH50006 Measure and Integration Lecture Notes*. Imperial College London, 2026.